

RefactoAI

Revamping Code Excellence through AI-Driven Code Refactoring

Project Team

Muhammad Nibras Aamir	21I-2683
Khadeeja Shah	21I-1653
Areeba Riaz	21I-1736

Session 2021-2025

Supervised by

Mr. Pir Sami Ullah Shah



Department of Computer Science

**National University of Computer and Emerging Sciences
Islamabad, Pakistan**

June, 2025

Contents

1	Introduction	1
1.1	Problem Statement	1
1.2	Scope	2
1.3	Modules	2
1.3.1	Module 1: Data Collection and Analysis	2
1.3.2	Module 2: Requirement Engineering	3
1.3.3	Module 3: Code Refactoring Utilizing Code-LLaMA	3
1.3.4	Module 4: Development of Front-End, Testing, and Performance Optimization	3
1.4	User Classes and Characteristics	4
2	Project Requirements	5
2.1	Use-case/Event Response Table/Storyboarding	5
2.2	Functional Requirements	7
2.2.1	Module 1: Code Upload and Validation	7
2.2.2	Module 2: User Story Integration	8
2.2.3	Module 3: Refactoring and Regeneration	8
2.2.4	Module 4: Testing and Reporting	8
2.3	Non-Functional Requirements	8
2.3.1	Reliability	8
2.3.2	Usability	9
2.3.3	Performance	9
2.3.4	Security	9
2.4	Domain Model	11
3	System Overview	13
3.1	Architectural Design	13
3.2	Design Models	15
3.3	Data Design	19
3.3.1	Major Data Entities and Structures	19
3.3.2	Data Flow and Processing Stages	20
3.3.3	Data Storage and Organization	21
3.3.4	Conclusion	21

References

23

List of Figures

2.1	Use Case Diagram	6
2.2	Domain Model Diagram	11
3.1	Architectural Diagram	14
3.2	Activity Diagram	15
3.3	State Sequence Diagram (SSD)	16
3.4	State Machine Diagram	17
3.5	Data Flow Diagram	18

List of Tables

1.1	User Classes and Characteristics for Fully Automated RefactoAI	4
2.1	Use-Case Table for Fully Automated RefactoAI	7
2.2	Product, Organizational, and External Requirements for Fully Automated RefactoAI	10

Chapter 1

Introduction

This document of project proposal is aimed at outlining the objectives, scope, and implementation plan for the project "RefactoAI– Revamping Code Excellence through AI Driven Refactoring" This document gives in detail the general view, background of the system, the problems that it intends to handle, and the solution that it sets forward. This project aims to come up with a web-based application that will facilitate the automation of refactoring Python code and regenerating the code using advanced large language models based on the requirements of the user.

1.1 Problem Statement

Under the condition of continuous change in modern software development, quality code is the essential underpinning to sustain applications over the long term [3]. The paper further considers that traditional code refactoring processes are of much manual effort, time-consuming, and thus are very sensitive to human error [5]. Evidently, from previous studies, it was shown that such manual efforts result in a lot of inefficiencies, increasing the rates of error and lowering both code quality and developer productivity levels [2]. This is due to the fact that a lot of time has to be spent on addressing quality, alignment, performance, and other issues in the code [1]. Research also suggests that bugs or other problems related to refactoring can increase the complexity of the development cycle [4]. Therefore, it is mandatory to come up with advanced automated solutions based on newly proposed techniques [6].

This project builds an automated solution that leverages advanced LLMs for refactoring and regenerating Python code from requirements. The purpose of the system is to facilitate the refactoring process itself, reducing the manual effort and enhancing the quality of the code. Receiving the requirements at the input of the refactoring process means that the system ensures results will be user-satisfying and project-expectation matching. It is

expected that this would save time, prevent errors, and improve the maintainability of the code in a whole, both in meeting practical developer needs and the theoretical challenges stated in existing research reports [4] [5] .

1.2 Scope

The project "RefactoAI—Revamping Code Excellence through AI-Driven Code Refactoring" tends to design and implement a web-based application using advanced Large Language Models, LLMs, like Code-LLaMA, GPT-NeoX, and others for automating Python refactoring. The main functionalities of the project are analyzing the existing Python code, inquiring what requirements to be added into the provided code, and then refactoring and regenerating the code by incorporating the requirements that have been given by the user, where an open-source LLM is used to convert user requirements into user stories, making sure that the regenerated code aligns with them.

RefactoAI also automates error testing and performance optimization, thus an all-in-one solution for Python developers. PyRat will be used in the last phase of the project to perform requirement analysis and traceability. The user stories and refactored code are passed via the tool for mapping into a detailed report showing how and where each requirement is mapped into implemented code. This project is limited to Python code and deals only with quality enhancement of the code and addition of requirements given by users into the code without going any deeper into other programming languages or broader processes of software development.

1.3 Modules

1.3.1 Module 1: Data Collection and Analysis

This module involves collecting and analyzing Python code that requires enhancement.

1. **Datasets Required:** Collect data from the following sources:
 - GeeksforGeeks Python projects
 - CopyAssignment Python programming projects
 - Project Euler solutions from GitHub repository
2. **Executing Tests Over the Script:** Execute test cases on the provided Python script by the user to identify defects, including issues related to alignment, indentation, or coding standards.

3. **Refactor the Code:** Clean up the code by improving alignment, removing extra indentations, and enhancing quality and best practices.
4. **Feedback & Suggestions:** Provide feedback on found errors in the code. Include detailed errors and suggestions for user code improvement.
5. **Design-Figma:** Develop Figma designs for the web application during the design phase.

1.3.2 Module 2: Requirement Engineering

This module involves decomposing user requirements into technical user stories using an open-source LLM.

1. **Gather Requirements:** Collect requirements from clients/users.
2. **Analyze and Synthesize:** Use an open-source LLM to analyze and synthesize the collected requirements into user stories in natural language.

1.3.3 Module 3: Code Refactoring Utilizing Code-LLaMA

This module involves refactoring and regenerating Python code based on recognized user stories.

1. **Convert User Stories:** Convert user stories into code requirements.
2. **Embed Use Cases:** Integrate the use cases into the original source code using Code-LLaMA.
3. **Verify Code:** Ensure that the refactored code meets all the requirements outlined in the user stories.

1.3.4 Module 4: Development of Front-End, Testing, and Performance Optimization

This module involves developing the user interface, optimizing performance, and performing final validation of the refactored code.

1. **Develop Web-Based Application Interface:** Create the user interface for the web application.
2. **Add Testing Tools:** Implement tools for automatic error detection and resolution.

3. **Optimize Refactored Code:** Maximize the performance of the refactored code.
4. Final Testing Using pyRat:
 - Add all scenarios, including user stories and refactored code, to the testing tool pyRat.
 - Obtain a report mapping each requirement to its relevant code.

1.4 User Classes and Characteristics

User Classes and Characteristics of RefactoAI

User Class	Description
User	A User is any individual who uses the system to upload Python code for refactoring and analysis. This could be a software engineer, developer, or a person without deep technical expertise. Users are expected to provide code and refactoring requirements. The system will guide them through the process of specifying their needs for code optimization and testing.
System (Auto-mated)	The system performs all backend processes autonomously. It uses large language models (LLMs) to refactor and regenerate Python code based on user inputs. The system handles code validation, error detection, automated testing, and performance optimization without any human intervention. It generates reports mapping user requirements to the refactored code.

Table 1.1: User Classes and Characteristics for Fully Automated RefactoAI

Chapter 2

Project Requirements

This chapter describes the functional and non-functional requirements of the project.

2.1 Use-case/Event Response Table/Storyboarding

- The **Use Case Diagram** for the RefactoAI system illustrates the complete workflow of how the user interacts with the system to refactor Python code using Large Language Models (LLMs). The process starts with the User uploading a Python file, which triggers the system to check the Python file for syntax errors. If the file contains errors (shown as a failure path), the system stops and requires the user to correct and re-upload the file. If the file passes validation, the system gathers user requirements and proceeds to generate user stories using an LLM. Based on the generated user stories, the system continues with code generation or refactoring through another LLM, and subsequently, the refactored code undergoes testing. If the generated code passes testing, a new Python file is created and returned to the user. If the code fails testing, it returns to the refactoring phase for further adjustments. In this diagram, the «include» relationship shows mandatory steps in the process, such as checking the file after it is uploaded. The «extend» relationship represents optional or conditional steps, like gathering user requirements extending the generation of user stories.

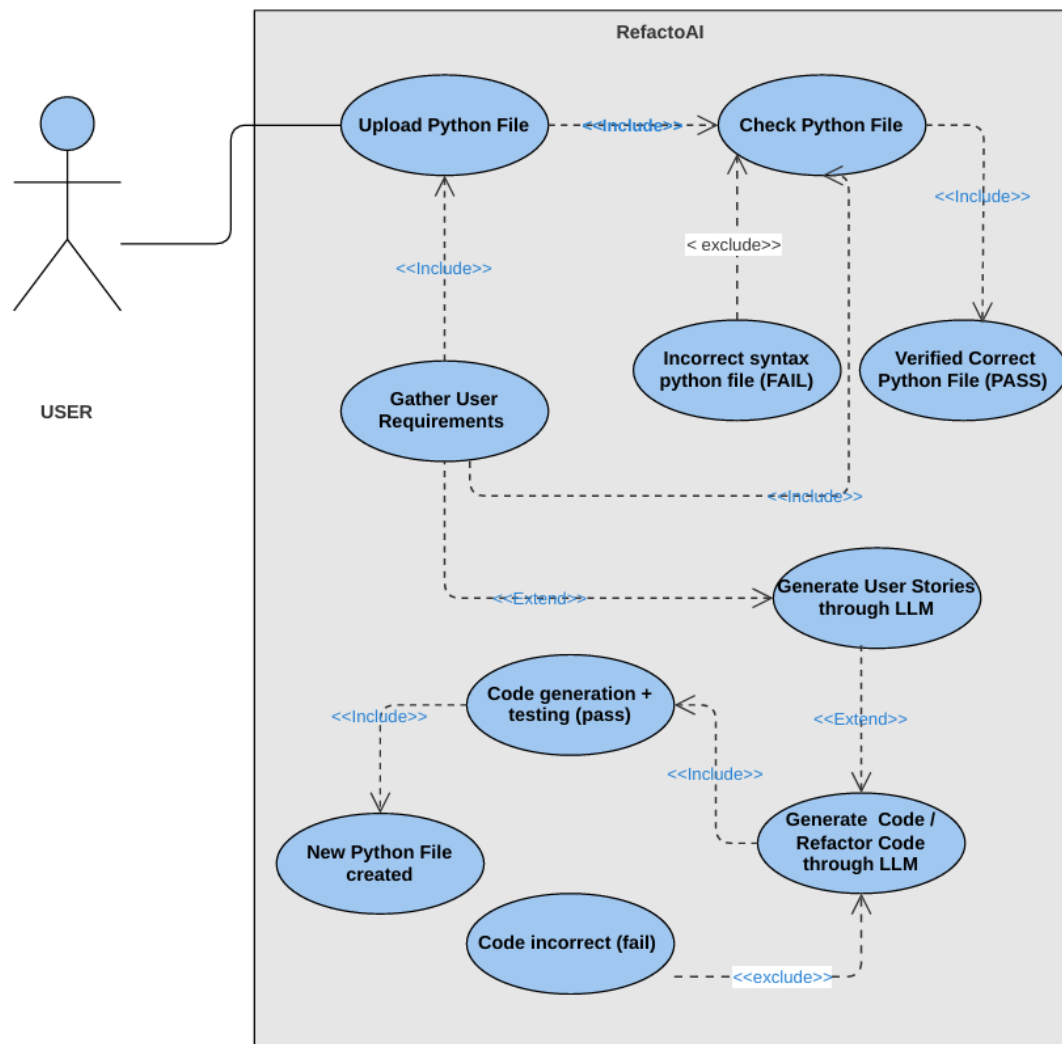


Figure 2.1: Use Case Diagram

Use Case	Actor	Description	System Response
Upload Code	User	The user uploads a Python script for refactoring.	The system automatically validates the code for errors (e.g., syntax, alignment) and prepares it for refactoring.
Input Requirements	User	The user specifies new requirements or user stories for the refactoring process.	The system processes these requirements using LLMs, refactors the code, and incorporates user-defined changes.
View Refactored Code	User	The user views the refactored code and its improvements.	The system displays the refactored code, including detailed changes and optimization suggestions.
Automated Testing	System	The system runs automated tests on the refactored code.	The system validates the code, ensuring it meets user stories and project requirements, and generates a report automatically.
Generate Report	System	The system generates a report on the refactoring process and the code's alignment with user requirements.	The system displays the report for the user, detailing the refactoring steps, test results, and final code status.

Table 2.1: Use-Case Table for Fully Automated RefactoAI

2.2 Functional Requirements

Below are the functional requirements of RefactoAI which will tell what the system is suppose to achieve.

2.2.1 Module 1: Code Upload and Validation

Following are the requirements for Module 1:

1. FR1: The system shall allow users to upload Python scripts for analysis.
2. FR2: The system shall validate the uploaded code for errors (syntax, indentation, etc.) before refactoring.
 - FR2.1: Upon error, the system shall ask the user to upload a new file.
 - FR2.2: If no errors are found, the system shall continue and move to the next step.

2.2.2 Module 2: User Story Integration

Following are the requirements for Module 2:

1. FR1: The system shall allow users to input requirements for the code.
2. FR2: The system shall analyze these requirements and translate them into user stories using open-source LLMs.

2.2.3 Module 3: Refactoring and Regeneration

Following are the requirements for Module 3:

1. FR1: The system shall refactor the uploaded code to improve its quality, performance, and maintainability.
2. FR2: The system shall regenerate the code by embedding the new user requirements into the existing code base.

2.2.4 Module 4: Testing and Reporting

Following are the requirements for Module 4:

1. FR1: The system shall validate the refactored code using pyRat or other testing tools.
2. FR2: The system shall generate a detailed report mapping user stories to the refactored code.

2.3 Non-Functional Requirements

This section specifies non-functional requirements.

2.3.1 Reliability

- Requirement: The system must operate without failure 90.5% of the time.
- Measurement: Mean time between failures (MTBF) should be at least 100 hours. The system must log any failures and automatically restart the refactoring process in case of an interruption.

2.3.2 Usability

- **Requirement:** The system should allow users to upload and refactor code with no more than 3 steps.
- **Example:** The interface shall be intuitive, allowing users to complete the refactoring process within 5 minutes of interaction.

2.3.3 Performance

- **Requirement:** The system must refactor the uploaded code within 5 seconds for small scripts (< 200 lines) and under 15 seconds for large scripts (up to 1000 lines).
- **Measurement:** Performance must be measured during peak hours when multiple users are refactoring code simultaneously.

2.3.4 Security

- **Requirement:** All user-uploaded code must be encrypted while stored in the system to protect intellectual property.
- **Measurement:** The system should resist unauthorized access with an intrusion detection mechanism that alerts developers of any suspicious activities.

Requirement Category	Subheading	Description
Product Requirements	Performance	The system must refactor code within 10 seconds for small scripts (<500 lines) and under 30 seconds for larger scripts (up to 2000 lines). Performance should be optimal under peak loads.
	Usability	The interface should be user-friendly, allowing users to upload code and view results with minimal steps (within 2 interactions).
	Reliability	The system should have an MTBF (Mean Time Between Failures) of over 300 hours and log any failures automatically. It must operate without failure 90.9% of the time.
	Scalability	The system should scale to support up to 100 simultaneous users and 500 code uploads per minute without performance degradation.
External Requirements	Compliance	The system must comply with data protection laws (e.g., GDPR) and ensure the security and confidentiality of user-uploaded code and data.
	Security	All uploaded code and data must be encrypted and stored securely. Unauthorized access attempts must be blocked, and incidents should be logged.
	External System Integration	The system must support integration with third-party tools (like LLM APIs) for code refactoring and testing functionalities (in the future).

Table 2.2: Product, Organizational, and External Requirements for Fully Automated RefactoAI

2.4 Domain Model

A **domain model** represents the key entities, their attributes, and relationships in a system, describing how they interact to fulfill the system's purpose. It provides a conceptual view of the real-world objects that exist in the system and the relationships between them. In the case of RefactoAI, the domain model would include entities such as the User, Python Script, LLM Models (for user story generation and code refactoring), requirements, and the testing tools. These entities work together to automate the code refactoring process from input (the Python script) to output (the refactored and tested code), based on the user's requirements and LLM driven processing.

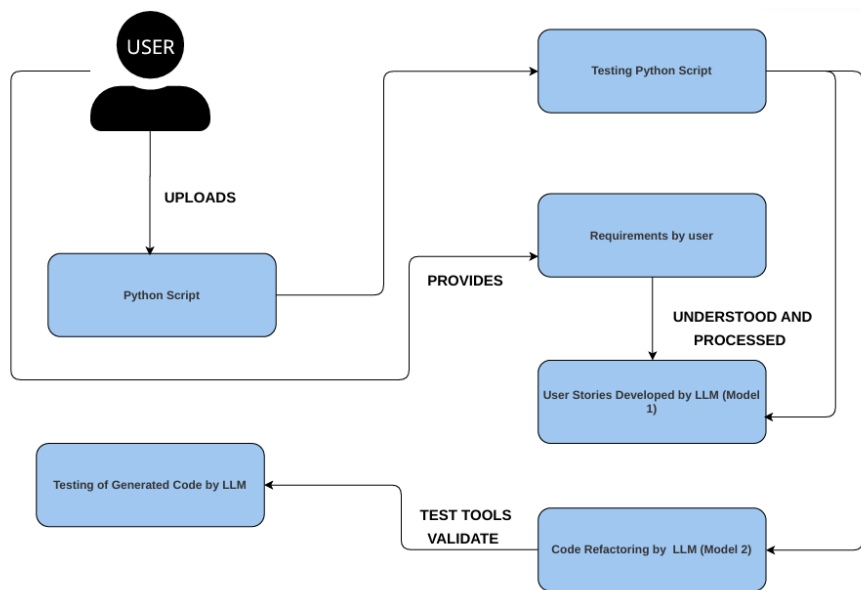


Figure 2.2: Domain Model Diagram

Chapter 3

System Overview

This project – RefactoAI – Revamping Code Excellence through AI-Driven Refactoring would be such a system that regenerates and improves the Python code using large language models. In this system, users can upload Python scripts, specify the requirements needed to be embedded and obtain refactored code with no human input. The focus is mainly on making the tedious work of code refactoring which is however manual and often full of errors, accurate and entirely automated in order to improve the quality, maintainability, and performance of the code with less interaction from the developer.

3.1 Architectural Design

- **User Interface (UI) Layer:**
 - The user uploads a Python file (.py) through the interface, initiating the refactoring process.
- **Application Layer (Logic):**
 - **Script Validation:** Ensures the uploaded script adheres to syntax rules.
 - **Generate User Stories:** User requirements are processed with an LLM to generate user stories stored in memory.
 - **Refactor Code:** Another LLM refactors the code based on a pre-trained dataset.
 - **Testing:** The refactored code is tested for performance and correctness.
- **File System Layer:**
 - Files (temporarily stored in a folder) hold the refactored code during processing.

- **Trained Data Layer:**

- The pre-trained dataset of 2 million Python scripts informs the LLM during code refactoring.

- **Conclusion:**

- The system uses file-based temporary storage to minimize computational overhead and improve efficiency during processing.

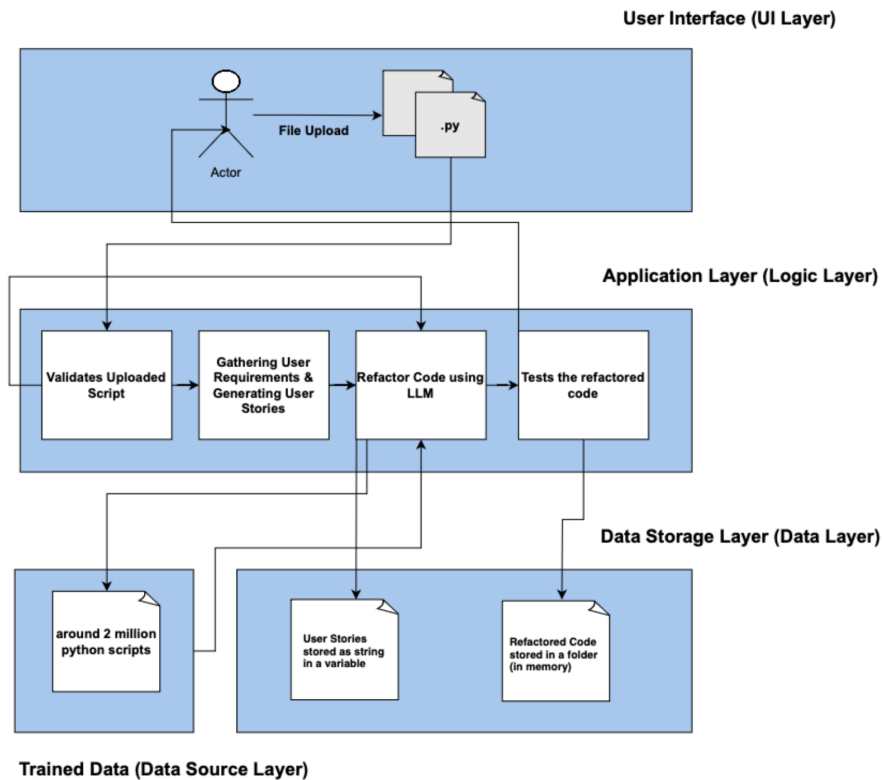


Figure 3.1: Architectural Diagram

3.2 Design Models

Design Models for Procedural Approach

- This **activity diagram** represents the workflow for the RefactoAI system, starting with the User uploading a Python file for refactoring. After the upload, the file undergoes validation to check for errors. If errors are found, the process moves to Requirement Specification, where the user can correct the requirements. Once the file passes validation, the system moves to User Stories generation using an LLM. The generated use cases or stories are then passed to the Code Generation/Refactoring phase, where another LLM refactors the Python code. After refactoring, the code is subjected to testing using different tools. If the code passes the testing phase, it is marked as verified code. If it fails the test, the process loops back to the Code Generation/Refactoring step, allowing for iterative improvements based on the test results.

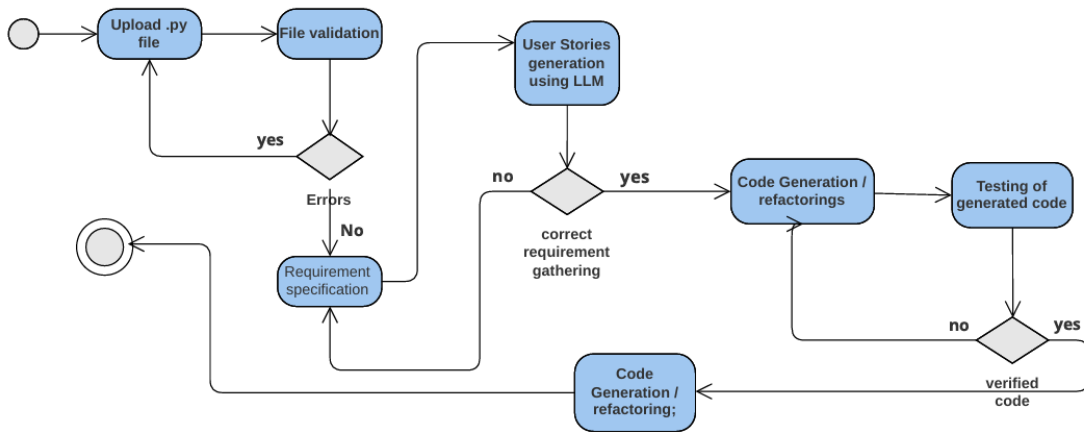


Figure 3.2: Activity Diagram

- The **System Sequence Diagram (SSD)** for RefactoAI outlines the interaction flow between the User and the System during the code refactoring process. The process starts with the User uploading a Python file, which is then validated by the system. If the validation returns errors, the system requests the user to correct the file and re-upload it. Once the file is valid, the user provides specific requirements, and the system uses LLMs to generate user stories. The generated user stories are then returned to the user. Following this, the system proceeds to the code refactoring phase using LLM, where it refactors or generates new code. The refactored code is tested, and if the tests pass, the system sends the verified code back to the user as a newly refactored Python file. The diagram illustrates this entire interaction and the flow of tasks from upload to the final delivery of the refactored Python file.

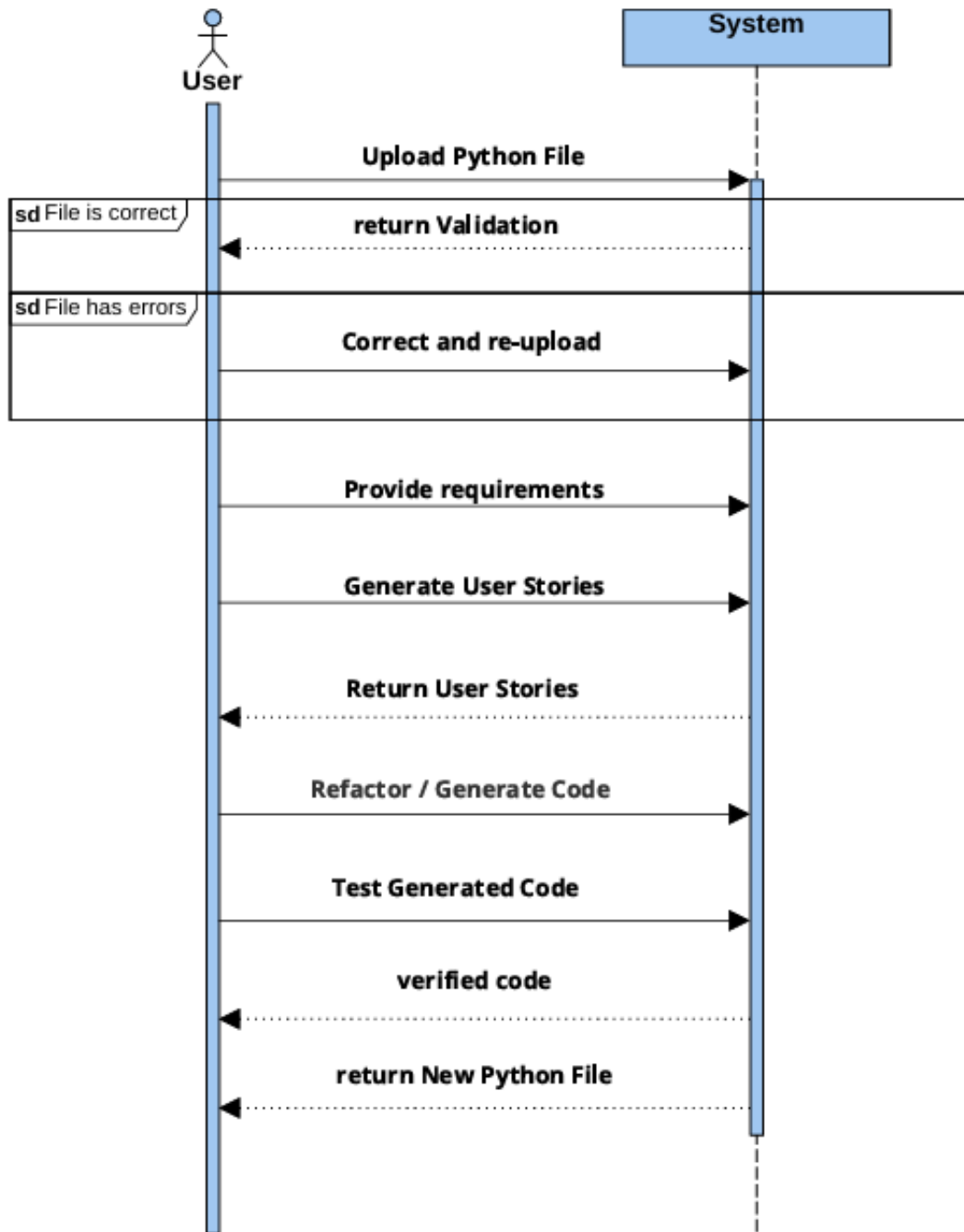


Figure 3.3: State Sequence Diagram (SSD)

- The **State Machine Diagram** for RefactoAI illustrates the progression of a Python script through various states, from upload to final delivery, with temporary file storage utilized at multiple stages to optimize the process. The system begins when the User Uploads the Python Script (Uploaded state). The script is then Validated for errors; if it fails, the user must upload a corrected version, repeating the cycle. If the script passes validation, the system Gathers Requirements, which are stored temporarily in the User Stories Storage (Files). After that, User Stories are Generated by the LLM, and the process moves to Code Refactoring using a pre trained dataset of Python scripts, temporarily storing the refactored code in Refactor Code Storage (Files). The refactored code is then Tested, and if it passes, the system creates a new Python file in the Finalized state, delivering it back to the user. If the test fails, the system loops back to refactor the code again. By using files temporarily, the system minimizes computational costs that would otherwise arise from a database. Temporary file storage allows for faster access and processing without the overhead of long-term data management, making the system efficient and scalable. Each state transition is triggered by successful or failed validations, user input, and LLM processing events.

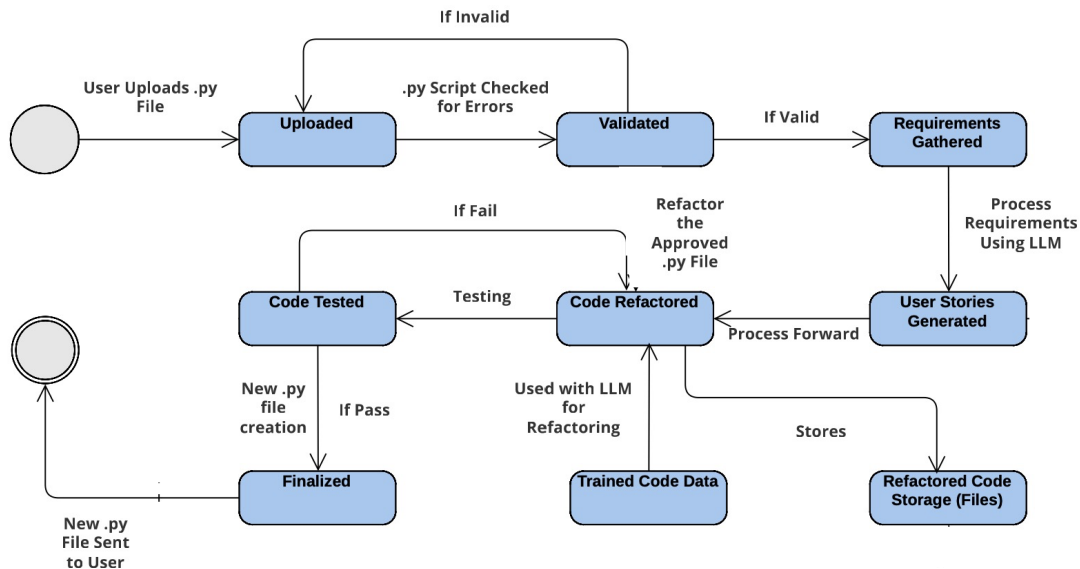


Figure 3.4: State Machine Diagram

- The **Data Flow Diagram (DFD)** for RefactoAI illustrates the systematic flow of data through various components of the system, highlighting how user inputs are transformed into outputs. The process begins with the user uploading a Python script, which triggers subsequent actions within the system. Initially, the uploaded script undergoes validation in the "Validate Python Script" phase, where it is checked

for correctness, and the user may be prompted for additional requirements. If the script is validated successfully, the system analyzes the requirements and converts them into user stories using a Language Model (LLM1). These user stories, along with the original script, are then processed in the "Refactor Code" phase, where a second Language Model (LLM2) is employed for refactoring and regeneration, enhancing the code's structure and efficiency. The refactored code is temporarily stored in the "Refactor Code Storage (Folder)" for further testing. In the "Test Refactored Code" phase, the system verifies the functionality of the refactored code. After successful testing, the final version of the code is sent back to the user, completing the data flow. This DFD emphasizes a file-based system for data storage, chosen for its efficiency and simplicity, as it minimizes overhead compared to traditional databases, ensuring smooth transitions of necessary data between processes.

Overall, it provides a clear representation of how data flows from the user through various stages of processing, storage, and output, optimizing the refactoring process within the RefactoAI system.

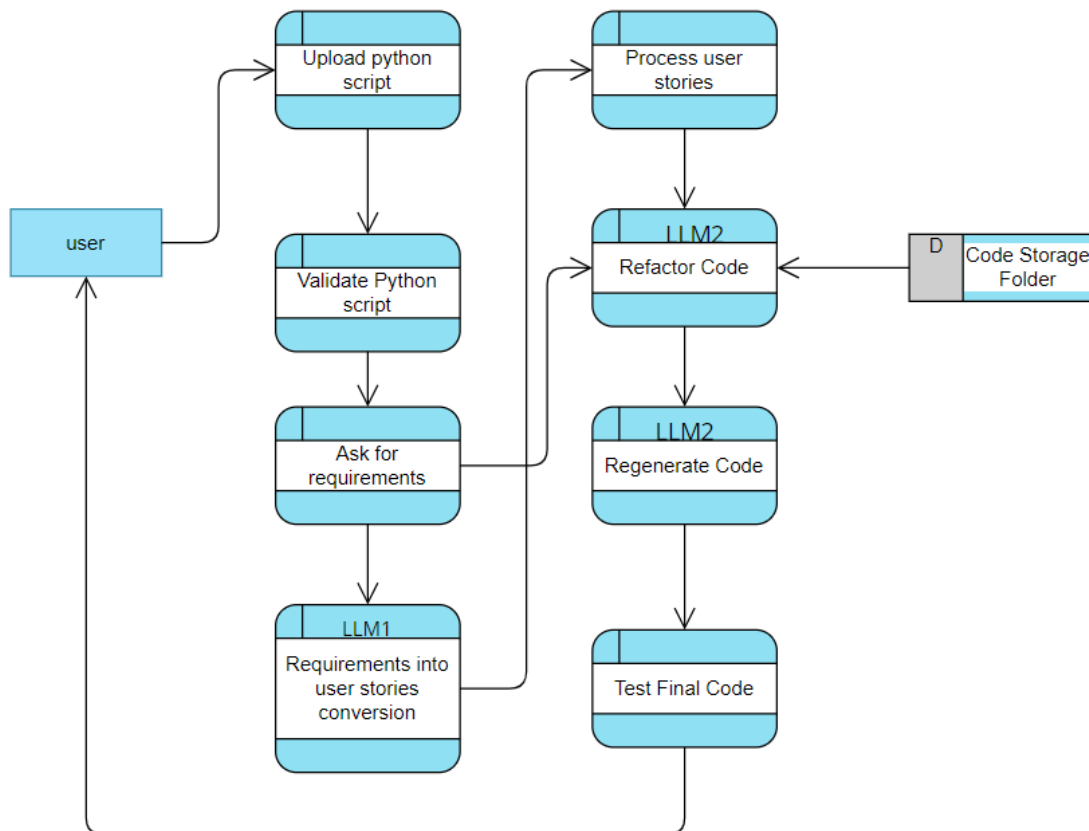


Figure 3.5: Data Flow Diagram

3.3 Data Design

The data design of RefactoAI outlines how data flows through the system, how it is processed, stored, and organized. The core entities involve the user-uploaded Python file, LLM-generated user stories, refactored and regenerating code, and the final report that links user requirements to specific code changes.

3.3.1 Major Data Entities and Structures

- **User-Uploaded Python File:**
 - **Data Structure:** The Python file is treated as a *file object*. The whole file is read into the system for analysis and refactoring.
 - **Storage:** The file is temporarily stored in a directory `/user_uploads/` until analysis and refactoring are complete.
 - **Processing:** The system runs tests on the file, including syntax checks, PEP8 compliance, and code quality analysis. If errors are found, the user is asked to upload a new file. If no errors are found, the system provides suggestions and a score for the code.
- **Test Results and Suggestions:**
 - **Data Structure:** Test results from syntax checks, PEP8 compliance, and complexity analysis are stored as *text output*.
 - **Storage:** These results are temporarily stored in memory and displayed to the user.
 - **Processing:** If no errors are found, suggestions and a score are generated. If errors are detected, the system prompts the user to upload a new file.
- **User Requirements:**
 - **Data Structure:** User requirements are entered as *text* and stored as a *list of strings*.
 - **Storage:** These requirements are stored temporarily in memory and used as input for LLM1.
 - **Processing:** The requirements are sent to **LLM1**, which converts them into user stories.
- **User Stories (LLM1 Output):**
 - **Data Structure:** The output from **LLM1** is a set of *text-based user stories*.

- **Storage:** These user stories are stored temporarily in memory and passed to the next stage for refactoring.
- **Processing:** The system passes the user stories to **LLM2**, which regenerates the code based on the user stories.
- **Refactored and Regenerated Code (LLM2 Output):**
 - **Data Structure:** The refactored and regenerated code is saved as a *file object*.
 - **Storage:** The new Python file is saved in the `/user_uploads/` directory.
 - **Processing:** **LLM2** refactors and regenerates the code based on the user stories. The final code is then tested to ensure no errors are introduced.
- **Final Report (Requirement-to-Code Mapping):**
 - **Data Structure:** The final report is structured as a *text file* or *JSON object* and links each part of the new code to corresponding user requirements.
 - **Storage:** The report is stored and made available for the user to view or download.
 - **Processing:** The report shows how the user requirements map to the specific parts of the refactored code.

3.3.2 Data Flow and Processing Stages

1. **User Uploads Python Code:** The user uploads the Python file, which is stored in a directory on the server. The system runs tests (syntax, PEP8, quality) on the file. If errors are detected, the user is prompted to upload a new file. If no errors are found, suggestions and a score are shown.
2. **User Inputs New Requirements:** Once the uploaded code passes the tests, the user inputs new requirements. These requirements are stored as a list of strings.
3. **LLM1: Convert Requirements to User Stories:** The user's requirements are sent to **LLM1**, which converts them into user stories. The stories are stored temporarily and passed to the next stage for refactoring.
4. **LLM2: Refactor Code Based on User Stories:** Both the original Python file and the user stories are passed to **LLM2**, which refactors and regenerates the code. The refactored code is saved in the `/user_uploads/` directory.
5. **Generate Requirement-to-Code Mapping Report:** The system generates a report that links user requirements to specific parts of the refactored code.

6. **Display Refactored Code and Report:** The refactored Python code and the final report are shown to the user, allowing them to review the changes and understand how their requirements were implemented.

3.3.3 Data Storage and Organization

- **File System:** Uploaded Python files and refactored versions are stored in a folder `/user_uploads/`.
- **In-Memory Data:** Temporary data, such as user requirements, LLM outputs (user stories and refactored code), and test results, are stored in memory during processing. The data is discarded after the final output is generated.
- **Report Generation:** The final report shows how user requirements are linked to specific parts of the refactored code. This report is saved in a structured format (text or JSON) and made available for download.

3.3.4 Conclusion

In RefactoAI, the data design transforms user inputs (Python files and requirements) into outputs (refactored code and reports) through various stages. Data is primarily stored in the file system, with intermediate steps processed in memory for efficiency. The final report provides a clear link between the user's requirements and the changes made in the code, allowing for transparency and understanding of the refactoring process.

Bibliography

- [1] ACM. Acm transactions on software engineering and methodology. *ACM Transactions on Software Engineering and Methodology*, 2024.
- [2] Capgemini. Automated testing efficiency. *Capgemini Insights*, 2024.
- [3] McKinsey & Company. It project budgets and code quality. *McKinsey Digital Insights*, 2024.
- [4] Ralph E. Johnson and Michele S. N. B. Refactoring-induced bugs. *Proceedings of the 12th International Conference on Software Maintenance*, 2012.
- [5] Miryung Kim. Developer time spent on refactoring. *IEEE Transactions on Software Engineering*, 2014.
- [6] OpenAI. Performance optimization. *XallyAI*, 2024.